Project Documentation
SpendWise

1.Introduction

- **Project Title:**Spendwise
- **Team ID:**SWTID-2026-2455
- **Team Leader:** Sabana Yasmine.N sabanayasmine87@gmail.com
- **Team Members:**

  Karunaa.D dkarunaa18@gmail.com

  Diya.A diya2006717@gmail.com

  Sofia.D sofiadevan2007@gmail.com

2.Project Overview:

SpendWise is a backend project I developed to learn how to build secure REST APIs using Node.js, Express, MongoDB, and JWT authentication. The main purpose of this project was to understand how backend systems work in real-world scenarios and how to manage user data safely while performing basic operations.

The project focuses on implementing user authentication and basic CRUD operations in a clean and simple way so that beginners can understand how backend development works.

Key Goals:

- Implement secure user login and signup using JWT.
- Perform basic CRUD operations (Create, Read, Update, Delete) on user transactions or records.
- Connect and interact with MongoDB using Mongoose.
- Learn how protected routes work using middleware.
- Keep the backend structure simple and beginner-friendly for easy learning.

Key Features / Functionality:

- Secure User Authentication: Ensures user accounts are protected using JWT tokens.
- Basic CRUD Operations: Users can add, view, update, and delete their transactions or records.
- MongoDB Integration: All data is stored and managed efficiently using Mongoose.
- Protected Routes: Certain endpoints are accessible only to authorized users.
- Beginner-Friendly Structure: Code is organized and easy to follow for learning purposes.

3.Project Instruction

SpendWise is a backend-only project built using Node.js and Express. Below are the steps to set up and run the project:

Quick Start:

1. Initialize the Node.js project using the command:

   npm init

2. Install all the required packages:

   npm install

3. Start the development server:

   npm run dev

Steps to Create the Project:

- Choose or create a directory for the backend project.
- Open terminal and navigate to that directory.
- Initialize Node.js project using npm init.
- Install required packages: express, mongoose, jsonwebtoken, bcrypt, cors, dotenv.
- Create folders: models, routes, controllers, middleware, config.
- Start the server using npm run dev.
- Test APIs using Postman or Thunder Client at http://localhost:5000.

4.Pre-requisites:

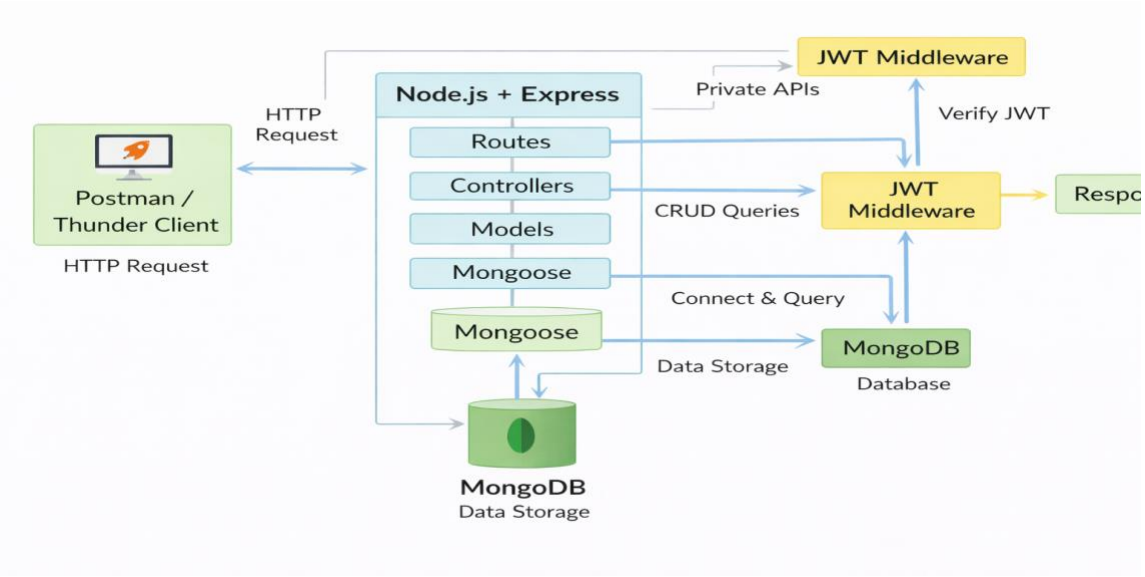
Software Requirements:

- Node.js – JavaScript runtime used to run the backend server.
- MongoDB – NoSQL database used for storing application data.
- npm – Package manager used to install and manage project dependencies.
- Git – Version control system used to track and manage code changes.
- Code Editor (VS Code) – Used to write, edit, and manage the project code.

Technical Requirements:

- JavaScript (ES6+) – Understanding modern JavaScript features such as arrow functions, promises, and async/await.
- Node.js and Express.js – Basic knowledge of creating backend servers and handling routes.
- MongoDB and Mongoose – Understanding how to store and manage data in a database.
- JWT Authentication – Basic idea of how JSON Web Tokens are used for user authentication.
- Basic REST API Concepts – Understanding HTTP methods such as GET, POST, PUT, and DELETE.
- Git Basics – Basic knowledge of version control to manage project code.

5.Project Architecture Image



6.Technical Architecture:

Backend Architecture (Node.js + Express)

- **Controllers:** Handle business logic for authentication and CRUD
- **Models:** Mongoose schemas for User and Transaction
- **Routes:** API endpoints for auth and CRUD operations
- **Middleware:** JWT verification and request protection
- **Configuration:** Database connection and environment variables

Database Architecture (MongoDB)

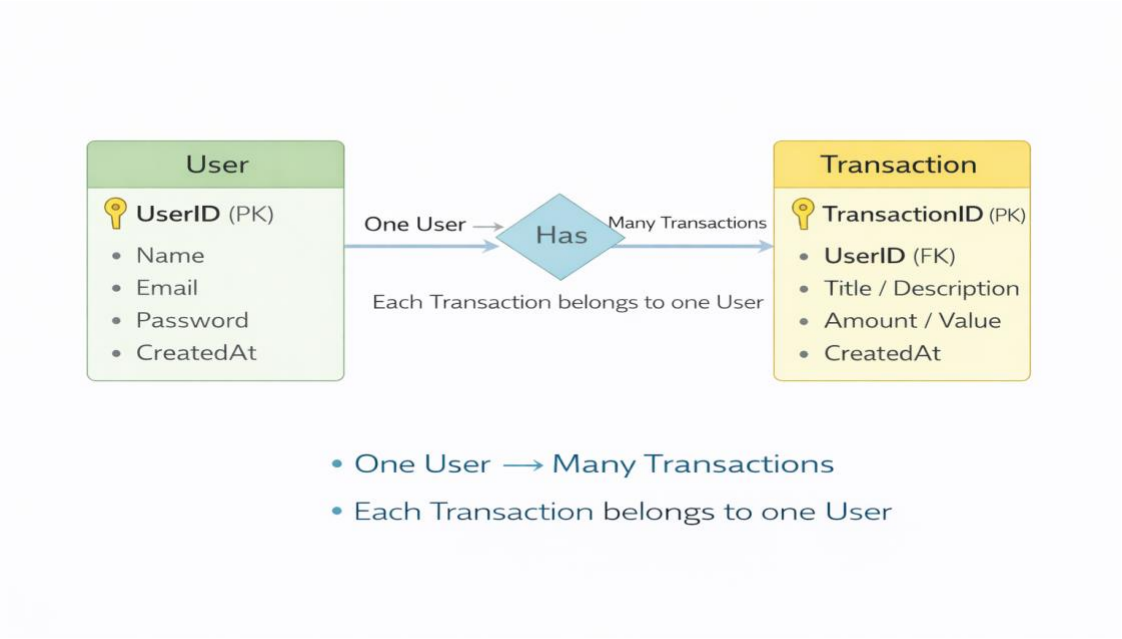
- Users Collection:

  _id, name, email, password (hashed), createdAt
- Transactions Collection:

  _id, userId (FK), title/description, amount/value, createdAt
- Relationship:

  Many Transactions belong to One User

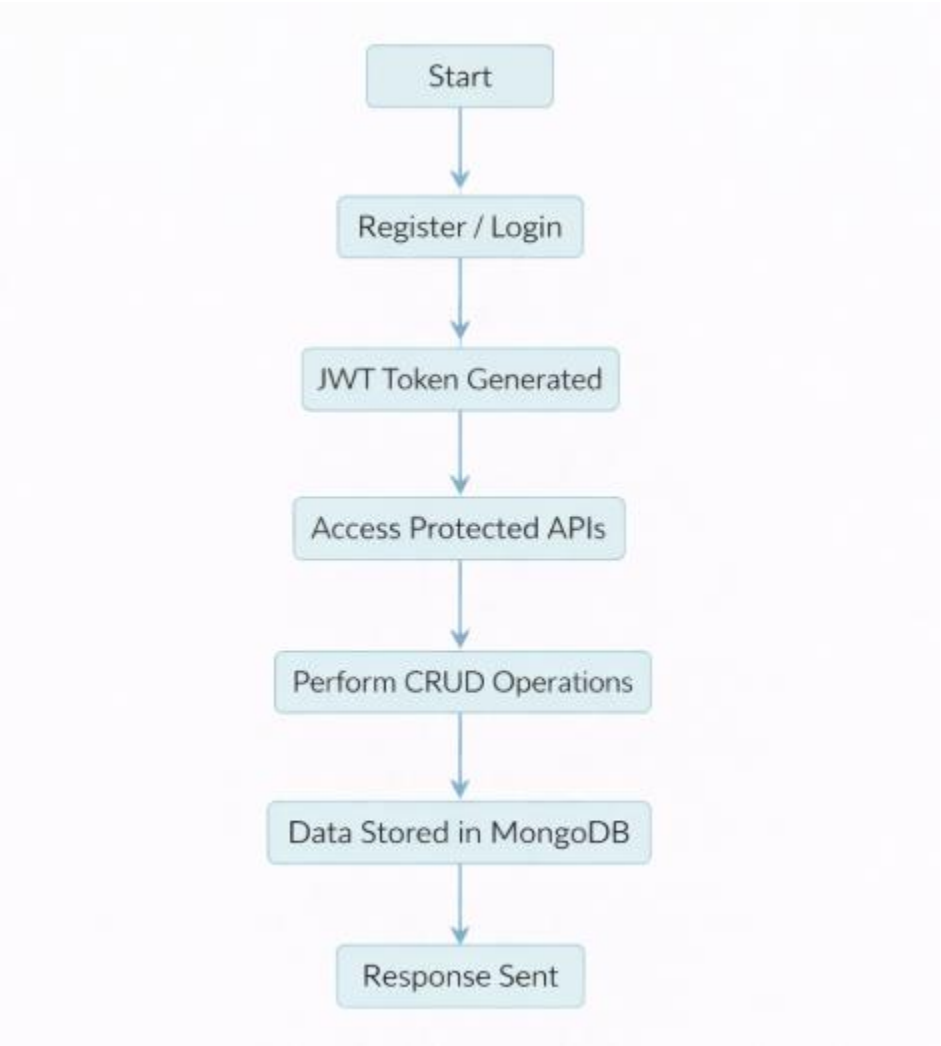
7.ER Diagram



8.Features

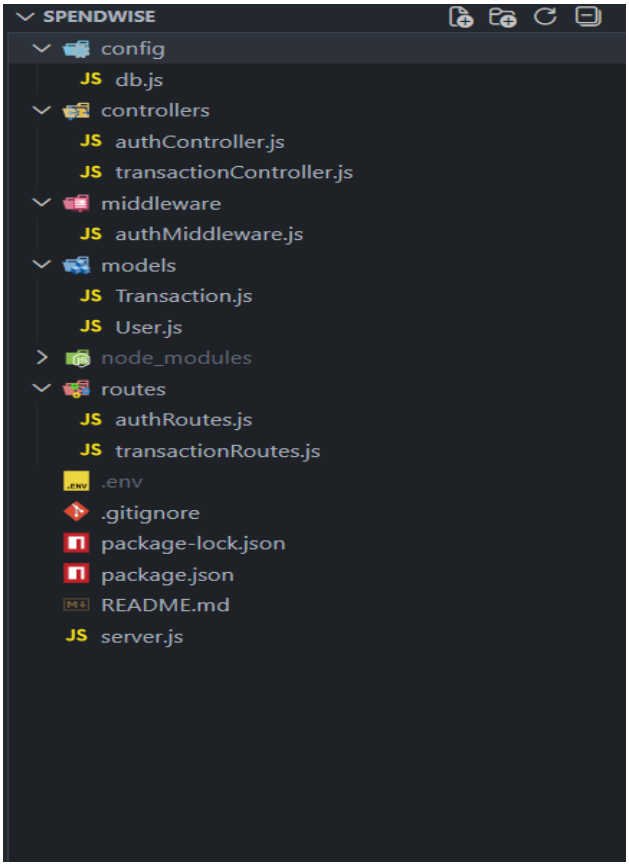
- User Authentication – Provides secure user registration and login using JWT authentication.
- CRUD Operations – Allows users to create, read, update, and delete transaction records.
- Protected Routes – Secures API endpoints using JWT middleware so that only authenticated users can access them.
- MongoDB Integration – Stores and retrieves data from MongoDB using Mongoose.

9.Users Flow:



10.PROJECT SETUP AND CONFIGURATION:

Folder Structure



11.Project Setup

1. Install required tools:

- Node.js
- MongoDB

2. Create project folders:
- server folders (models, routes, controllers, middleware, config)

3. Install Backend Packages:
- Express
- Mongoose
- Cors
- Jsonwebtoken
- Bcrypt
- Dotenv

12.BACKEND DEVELOPMENT

- Setup Express server
- Create server.js file
- Create .env file and define PORT and DB URL

```env
PORT=5000
MONGO_URI=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
JWT_SECRET=your_secret_key
```

- Configure middleware (cors, express.json)

- User Authentication:

  - Create Register and Login routes
  - Generate JWT token
  - Protect routes using middleware

- Define API Routes:

  - Auth routes

```node
const router = express.Router();

router.post("/register", registerUser);
router.post("/login", loginUser);

module.exports = router;
```

  - Transaction CRUD routes

- Implement Data Models:
    - User schema

```
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
      type: String,
      required: true,
      minlength: 6,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

    - Transaction schema

```
const transactionSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    amount: {
      type: Number,
      required: true,
    },
    type: {
      type: String,
      enum: ["income", "expense"],
      required: true,
    },
    category: {
      type: String,
      default: "General",
    },
    note: {
      type: String,
    },
    date: {
      type: Date,
      default: Date.now,
    },
  },
  { timestamps: true }
);
```

- Implement CRUD operations

13.DATABASE DEVELOPMENT

Configure MongoDB

- Use MongoDB Atlas or local MongoDB
- Store connection URL in .env file
- Connect using Mongoose

Create Database Connection

- Create a config file for DB connection

```
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

```
config > JS db.js > ...
const mongoose = require("mongoose");

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGO_URI);
        console.log(`MongoDB Connected: ${conn.connection.host}`);
    } catch (error) {
        console.error("MongoDB connection error:", error.message);
        process.exit(1);
    }
};

module.exports = connectDB;
```

- Use mongoose.connect()
- Handle success and error cases

14.Create Schema and Models

User Model:

- Name
- email
- password

Data / Transaction Model:

- userId
- title / description
- amount / value
- timestamps

15.PROJECT EXECUTION

Project Execution Steps:

1. Start MongoDB
2. Run backend server

```
found 0 vulnerabilities
PS C:\Users\a\Spendwise\spendwise> npm run dev

> spendwise@1.0.0 dev
> nodemon server.js


[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙ load multiple .env files [nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙ load multiple .env files [dotenv@17.3.1] injecting env (3) from .env
 with { path: ['.env.local', '.env'] }  with { path: ['.env.local', '.env'] }
Server running on port 5000
MongoDB Connected: ac-gw7oo4g-shard-00-01.6hjnhds.mongodb.net
```

3. Test Register API

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "name": "Sabana yasmine",
  "email": "sabana@gmail.com",
  "password": "project$spendwisepassword"
}
```

Response Headers Cookies Results Docs {}


```
{
  "_id": "69ad6c553573414b314072fc",
  "name": "Sabana yasmine",
  "email": "sabana@gmail.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6IjY5YWQ2YzU1MzU3MzQxNGIzMTQwNzJmYyIsImlhdCI
      6MTc3Mjk3MzE0MSwiZXhwIjoxNzczNTc3OTQxfQ.jQkOFI8_HDn
      -1mFKQO6L4j3-JvqEo6TwCMhJCmUJTqo"
}
```


4. Test Login API

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "email": "sabana@gmail.com",
  "password": "project$spendwisepassword"
}
```


```
{
  "_id": "69ad6c553573414b314072fc",
  "name": "Sabana yasmine",
  "email": "sabana@gmail.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6IjY5YWQ2YzU1MzU3MzQxNGIzMTQwNzJmYyIsImlhdCI
      6MTc3Mjk3MzIwMiwiZXhwIjoxNzczNTc4MDAyfQ
      .kQAs6cbe1QOh8F9GUa_ZsktqqWea-doeCzJW6ZtMLCI"
}
```


5. Copy JWT token


```
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
    .eyJpZCI6IjY5YWQ2YzU1MzU3MzQxNGIzMTQwNzJmYyIsImlhdCI
    6MTc3Mjk3MzIwMiwiZXhwIjoxNzczNTc4MDAyfQ
```


6. Use token to access protected CRUD APIs

7. Perform Create, Read, Update, Delete operations

16.OUTPUT SCREENSHOT

1. Register API

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "name": "Sabana yasmine",
  "email": "sabana@gmail.com",
  "password": "project$spendwisepassword"
}
```

```
{
  "_id": "69ad6c553573414b314072fc",
  "name": "Sabana yasmine",
  "email": "sabana@gmail.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6IjY5YWQ2YzU1MzU3MzQxNGIzMTQwNzJmYyIsImlhdCI
      6MTc3Mjk3MzE0MSwiZXhwIjoxNzczNTc3OTQxfQ.jQkOFI8_HDn
      -1mFKQO6L4j3-JvqEo6TwCMhJCmUJTqo"
}
```

2. Login API

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "email": "sabana@gmail.com",
  "password": "project$spendwisepassword"
}
```

```
{
  "_id": "69ad6c553573414b314072fc",
  "name": "Sabana yasmine",
  "email": "sabana@gmail.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6IjY5YWQ2YzU1MzU3MzQxNGIzMTQwNzJmYyIsImlhdCI
      6MTc3Mjk3MzIwMiwiZXhwIjoxNzczNTc4MDAyfQ
      .kQAs6cbe1QOh8F9GUa_ZsktqqWea-doeCzJW6ZtMLCI"
}
```

3. Create Transaction

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "type": "expense",
  "amount": 300,
  "category": "Food",
  "note": "Dinner"
}
```

```
{
  "user": "69ac26ce44d810441d9cc105",
  "amount": 300,
  "type": "expense",
  "category": "Food",
  "note": "Dinner",
  "date": "2026-03-08T12:11:55.787Z",
  "_id": "69ad678b3573414b314072dd",
  "createdAt": "2026-03-08T12:11:55.799Z",
  "updatedAt": "2026-03-08T12:11:55.799Z",
  "__v": 0
}
```

4. Get information

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
```

Copy

```
2   {
3     "_id": "69ad678b3573414b314072dd",
4     "user": "69ac26ce44d810441d9cc105",
5     "amount": 300,
6     "type": "expense",
7     "category": "Food",
8     "note": "Dinner",
9     "date": "2026-03-08T12:11:55.787Z",
10    "createdAt": "2026-03-08T12:11:55.799Z",
11    "updatedAt": "2026-03-08T12:11:55.799Z",
12    "__v": 0
13  },
14  {
15    "_id": "69ac42550ee4526b894e60df",
16    "user": "69ac26ce44d810441d9cc105",
17    "amount": 300,
18    "type": "expense",
```

5. Update API

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "amount": 400,
3   "type": "expense",
4   "category": "Shopping",
5   "note": "Groceries",
6   "date": "2026-03-07"
7 }
```

Copy

```
1 {
2   "_id": "69ac345344d810441d9cc13a",
3   "user": "69ac26ce44d810441d9cc105",
4   "amount": 400,
5   "type": "expense",
6   "category": "Shopping",
7   "note": "Groceries",
8   "date": "2026-03-07T00:00:00.000Z",
9   "createdAt": "2026-03-07T14:21:07.396Z",
10  "updatedAt": "2026-03-08T12:13:08.298Z",
11  "__v": 0
12 }
```

6. Delete API

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
```

```
1 {
2   "message": "Transaction removed"
3 }
```

[Project Demo Video]